

NAMA : AHMAD SYAFIQ MAULANA ZUHRI
NIM : 112110080
KELAS : 3C
MATA KULIAH : ANALISA ALGORITMA
DOSEN PENGAMPU : M. HASAN WAHYUDI, S.KOM,.ST

QUIZ 2

(Perbandingan Algoritma Brute Force dan Backtracking dalam Penyelesaian Sudoku)

TUGAS

- Implementasikan dalam bahasa pemrograman yang kalian mengerti (Java, PHP, Python, dll)
- Analisa perbandingan dua algoritma tersebut

JAWABAN

- Implementasi Bahasa Java

- Brute force

➤ Input

```
public class BruteForce {  
  
    public boolean isSafe(int[][] b, int r, int c, int n) {  
        // loop untuk menangani bentrokkan di deretan grid  
        for (int d = 0; d < b.length; d++) {  
            // jika angka yang kita masukkan sudah  
            // ada di baris itu maka kembalikan false  
            if (b[r][d] == n) {  
                return false;  
            }  
        }  
        // loop untuk menangani bentrokkan di deretan grid  
        for (int r1 = 0; r1 < b.length; r1++) {  
            // jika angka yang kita masukkan sudah  
            // ada di baris itu maka kembalikan false  
            if (b[r1][c] == n) {  
                return false;  
            }  
        }  
        // perulangan menangani bentrokkan dalam sub-grid yang ada dalam grid  
        int sqt = (int) Math.sqrt(b.length);  
        int boxRowSt = r - r % sqt;  
        int boxColSt = c - c % sqt;  
  
        for (int r1 = boxRowSt; r1 < boxRowSt + sqt; r1++) {  
            for (int d = boxColSt; d < boxColSt + sqt; d++) {  
                // jika angka yang kita masukkan sudah  
                // ada di baris itu maka kembalikan false
```

```

        if (b[r1][d] == n) {
            return false;
        }
    }
}

// jika tidak ada bentrokan dalam grid, maka aman dan
// benar dikembalikan
return true;
}

public boolean solveSudoku(int[][] b, int num) {
    int r = -1;
    int c = -1;
    boolean isVacant = true;
    for (int i = 0; i < num; i++) {
        for (int j = 0; j < num; j++) {
            if (b[i][j] == 0) {
                r = i;
                c = j;
            }
        }
    }

    // nilai salah berarti
    // masih ada beberapa
    // sel kosong di dalam kisi
    isVacant = false;
    break;
}

if (!isVacant) {
    break;
}

// tidak ada ruang kosong yang tersisa
// di dalam kisi-kisi
if (isVacant) {
    return true;
}

// jika tidak untuk setiap baris lakukan backtracking
for (int no = 1; no <= num; no++) {
    if (isSafe(b, r, c, no)) {
        b[r][c] = no;
        if (solveSudoku(b, num)) {
            // tampilan(board, num);
            return true;
        } else {
            b[r][c] = 0;
        }
    }
}

return false;
}

```

```

        public void display(int[][] b, int n) {
// Kita sudah mendapatkan solusinya, tampilkan saja
        for (int i = 0; i < n; i++) {
            for (int d = 0; d < n; d++) {
                System.out.print(b[i][d]);
                System.out.print(" ");
            }

            System.out.print("\n");
            if ((i + 1) % (int) Math.sqrt(n) == 0) {
                System.out.print("");
            }
        }
    }

// main method
    public static void main(String argsv[]) {
        long start = System.nanoTime();
// the 9 x 9 grid
        int[][] b = new int[][]{{1, 0, 0, 0, 0, 0, 7, 0, 0},
            {4, 0, 6, 0, 0, 0, 0, 0, 3},
            {0, 0, 0, 1, 0, 3, 0, 0, 0},
            {2, 0, 0, 3, 6, 0, 0, 9, 7},
            {0, 0, 5, 0, 9, 0, 2, 0, 0},
            {8, 9, 0, 0, 1, 4, 0, 0, 5},
            {0, 0, 0, 6, 0, 2, 0, 0, 0},
            {6, 0, 0, 0, 0, 0, 5, 0, 1},
            {0, 0, 8, 0, 0, 0, 0, 0, 2}};
// membuat objek dari kelas SudokuPuzzle
        BruteForce obj = new BruteForce();
// mneghitung ukuran grid
        int size = b.length;
        System.out.println("The grid is: ");
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                System.out.print(b[i][j] + " ");
            }
            System.out.println();
        }
        System.out.println();
        if (obj.solveSudoku(b, size)) {
// display solution

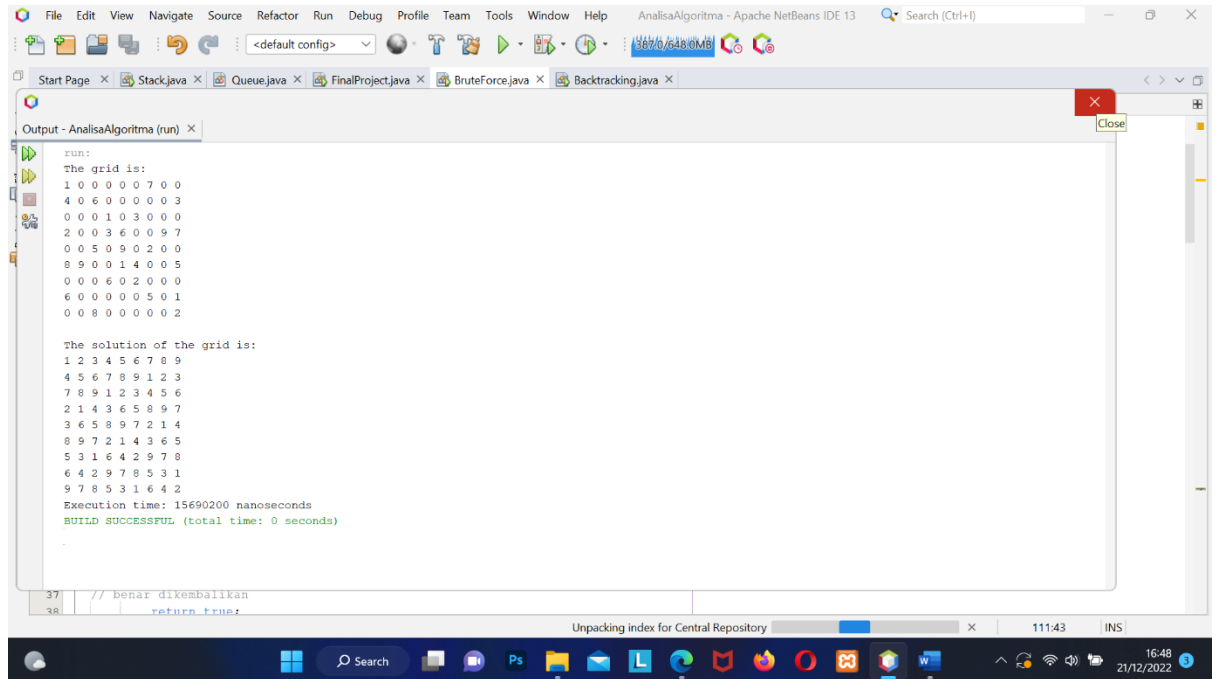
            System.out.println("The solution of the grid is: ");
            obj.display(b, size);
        } else {
            System.out.println("There is no solution available.");
        }
        long end = System.nanoTime();
        long execution = end - start;
        System.out.println("Execution time: " + execution + " nanoseconds");
    }
}

```

}

}

➤ Output



```
run:
The grid is:
1 0 0 0 0 0 7 0 0
4 0 6 0 0 0 0 0 3
0 0 0 1 0 3 0 0 0
2 0 0 3 6 0 0 9 7
0 0 5 0 9 0 2 0 0
8 9 0 0 1 4 0 0 5
0 0 0 6 0 2 0 0 0
6 0 0 0 0 0 5 0 1
0 0 8 0 0 0 0 0 2

The solution of the grid is:
1 2 3 4 5 6 7 8 9
4 5 6 7 8 9 1 2 3
7 8 9 1 2 3 4 5 6
2 1 4 3 6 5 8 9 7
3 6 5 8 9 7 2 1 4
8 9 7 2 1 4 3 6 5
5 3 1 6 4 2 9 7 8
6 4 2 9 7 8 5 3 1
9 7 8 5 3 1 6 4 2

Execution time: 15690200 nanoseconds
BUILD SUCCESSFUL (total time: 0 seconds)
```

- Backtracking
 - Input

```
public class Backtracking {

    private static int[][] grid = new int[9][9];

    public static void main(String[] args) {
        long start = System.nanoTime();
        solveS();
        displaySudoku();

        long end = System.nanoTime();
        long execution = end - start;
        System.out.println("Execution time: " + execution + " nanoseconds");
    }

    public static void displaySudoku() {
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[0].length; j++) {
                System.out.print(" " + grid[i][j] + " ");
            }

            System.out.println();
        }
    }
}
```

```

public static boolean solveS() {
    for (int row = 0; row < grid.length; row++) {
        for (int col = 0; col < grid[0].length; col++) {
            if (grid[row][col] == 0) {
                for (int number = 1; number <= 9; number++) {
                    if (!checkQuadrant(row, col, number)
                        && !checkCol(row, col, number) && !checkRow(row,
col, number)) {
                        grid[row][col] = number;
                        boolean sol = solveS();

                        if (sol) {
                            return true;
                        } else {

                            grid[row][col] = 0;
                        }
                    }
                }
            }
            return false;
        }
    }
    return true;
}

```

```

private static boolean checkRow(int row, int col, int value) {
    int c = row;
    for (int x = 0; x < grid.length; x++) {
        if (grid[c][x] == value) {
            return true;
        }
    }
    return false;
}

```

```

private static boolean checkCol(int row, int col, int value) {
    int c = col;
    for (int x = 0; x < grid.length; x++) {
        if (grid[x][c] == value) {
            return true;
        }
    }
    return false;
}

```

```

private static boolean checkQuadrant(int row, int col, int value) {
    if (row >= 0 && row <= 2) {
        //periksa kuadran pertama
    }
}

```

```

    if (col >= 0 && col <= 2) {
        for (int x = 0; x <= 2; x++) {
            for (int j = 0; j <= 2; j++) {
                if (grid[x][j] == value) {
                    return true;
                }
            }
        }
        return false;
    } //periksa kuadran kedua
    else if (col >= 3 && col <= 5) {
        for (int x = 0; x <= 2; x++) {
            for (int j = 3; j <= 5; j++) {
                if (grid[x][j] == value) {
                    return true;
                }
            }
        }
        return false;
    } //periksa kuadran ketiga
    else if (col >= 6 && col <= 8) {
        for (int x = 0; x <= 2; x++) {
            for (int j = 6; j <= 8; j++) {
                if (grid[x][j] == value) {
                    return true;
                }
            }
        }
        return false;
    }
    } else if (row >= 3 && row <= 5) {
//periksa kuadran keempat
        if (col >= 0 && col <= 2) {
            for (int x = 3; x <= 5; x++) {
                for (int j = 0; j <= 2; j++) {

                    if (grid[x][j] == value) {
                        return true;
                    }
                }
            }
            return false;
        } //periksa kuadran kelima
        else if (col >= 3 && col <= 5) {
            for (int x = 3; x <= 5; x++) {
                for (int j = 3; j <= 5; j++) {
                    if (grid[x][j] == value) {
                        return true;
                    }
                }
            }
        }
    }
}

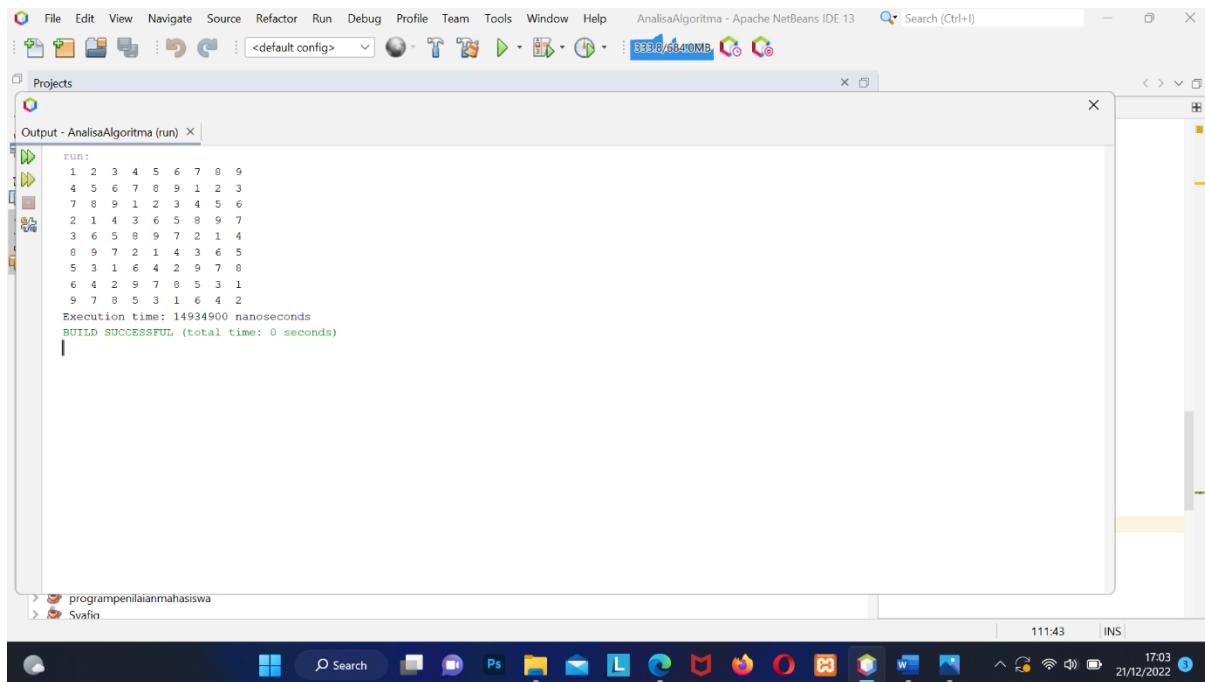
```

```

        return false;
    } //periksa kuadran keenam
    else if (col >= 6 && col <= 8) {
        for (int x = 3; x <= 5; x++) {
            for (int j = 6; j <= 8; j++) {
                if (grid[x][j] == value) {
                    return true;
                }
            }
        }
        return false;
    }
} else if (row >= 6 && row <= 8) {
//periksa kuadran ketujuh
    if (col >= 0 && col <= 2) {
        for (int x = 6; x <= 8; x++) {
            for (int j = 0; j <= 2; j++) {
                if (grid[x][j] == value) {
                    return true;
                }
            }
        }
        return false;
    } //periksa kuadran kedelapan
    else if (col >= 3 && col <= 5) {
        for (int x = 6; x <= 8; x++) {
            for (int j = 3; j <= 5; j++) {
                if (grid[x][j] == value) {
                    return true;
                }
            }
        }
        return false;
    } //periksa kuadran kesembilan
    else if (col >= 6 && col <= 8) {
        for (int x = 6; x <= 8; x++) {
            for (int j = 6; j <= 8; j++) {
                if (grid[x][j] == value) {
                    return true;
                }
            }
        }
        return false;
    }
}
return false;
}
}

```

➤ Output



```
run:
1 2 3 4 5 6 7 8 9
4 5 6 7 8 9 1 2 3
7 8 9 1 2 3 4 5 6
2 1 4 3 6 5 8 9 7
3 6 5 8 9 7 2 1 4
8 9 7 2 1 4 3 6 5
5 3 1 6 4 2 9 7 8
6 4 2 9 7 8 5 3 1
9 7 8 5 3 1 6 4 2
Execution time: 14934900 nanoseconds
BUILD SUCCESSFUL (total time: 0 seconds)
```

b. Analisa perbandingan kompleksitas waktu antara dua algoritma

Dalam menganalisis kompleksitas waktu, kita menghitung berapa banyak operasi yang dilakukan oleh algoritma. Sedangkan dalam analisis kompleksitas ruang, kita menghitung seberapa besar ruang yang diperlukan oleh algoritma. Untuk algoritma brute force, dilakukan pengisian kotak sebanyak n kali untuk m kotak kosong. Selain itu, dilakukan pengecekan apakah angka pada masing-masing kotak muncul tepat satu kali dalam baris, kolom, dan submatriks yang sama sehingga pengecekan dilakukan sebanyak n^8 untuk n^7 elemen. Dan untuk Pengecekan waktu dari brute force sendiri yaitu 6.387.800 nanoseconds.

Untuk algoritma Backtracking dilakukan pengisian m kotak yang kosong sebanyak n kali. Kemudian, pengecekan hanya dilakukan untuk kotak yang kosong sebanyak $n^8 \times m$ kali. Selain itu, algoritma melakukan iterasi satu-satu untuk mencari kotak yang kosong sebanyak n^7 . Untuk pengecekan waktu dari Backtracking sendiri yaitu 4.048.900 nanoseconds.

Kesimpulannya Didapatkan bahwa kompleksitas waktu algoritma backtracking lebih baik dibandingkan dengan algoritma brute force meskipun perbedaanya tidak terlalu jauh. Kompleksitas waktu kedua algoritma akan sama untuk kasus terburuk.